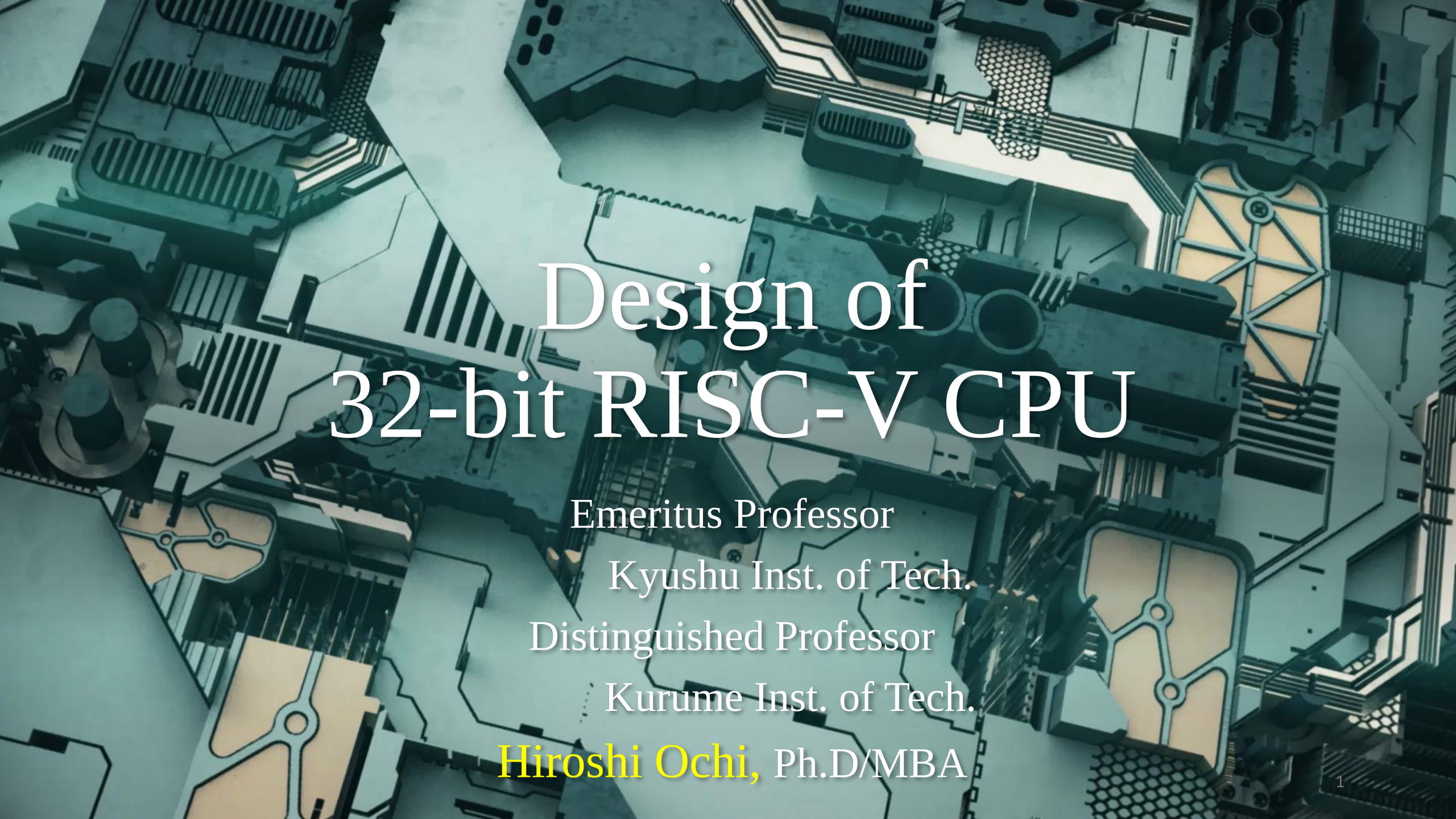


Design of 32-bit RISC-V CPU

Emeritus Professor

Kyushu Inst. of Tech.

Distinguished Professor

Kurume Inst. of Tech.

Hiroshi Ochi, Ph.D/MBA

Copy right

- All figures and code are from
  “Perfect Introduction of Computer Architecture RISC-V”
  by Prof.Kenji Kise of Tokyo Inst. of Tech.
  Gijyutu Hyoron Sha pub. Com.

Agenda

1-3	What is the RISC CPU?
4-1～4-8	Instruction types
5-1～5-2	Single Cycle Processor Components
5-3	Adder only
5-4	Register file
5-5～5-6	Immediate value, load, store, branch instructions

Review on 2's comp. with bit extension

Q1. Find decimal value of 4'b1110 and 5'b11110

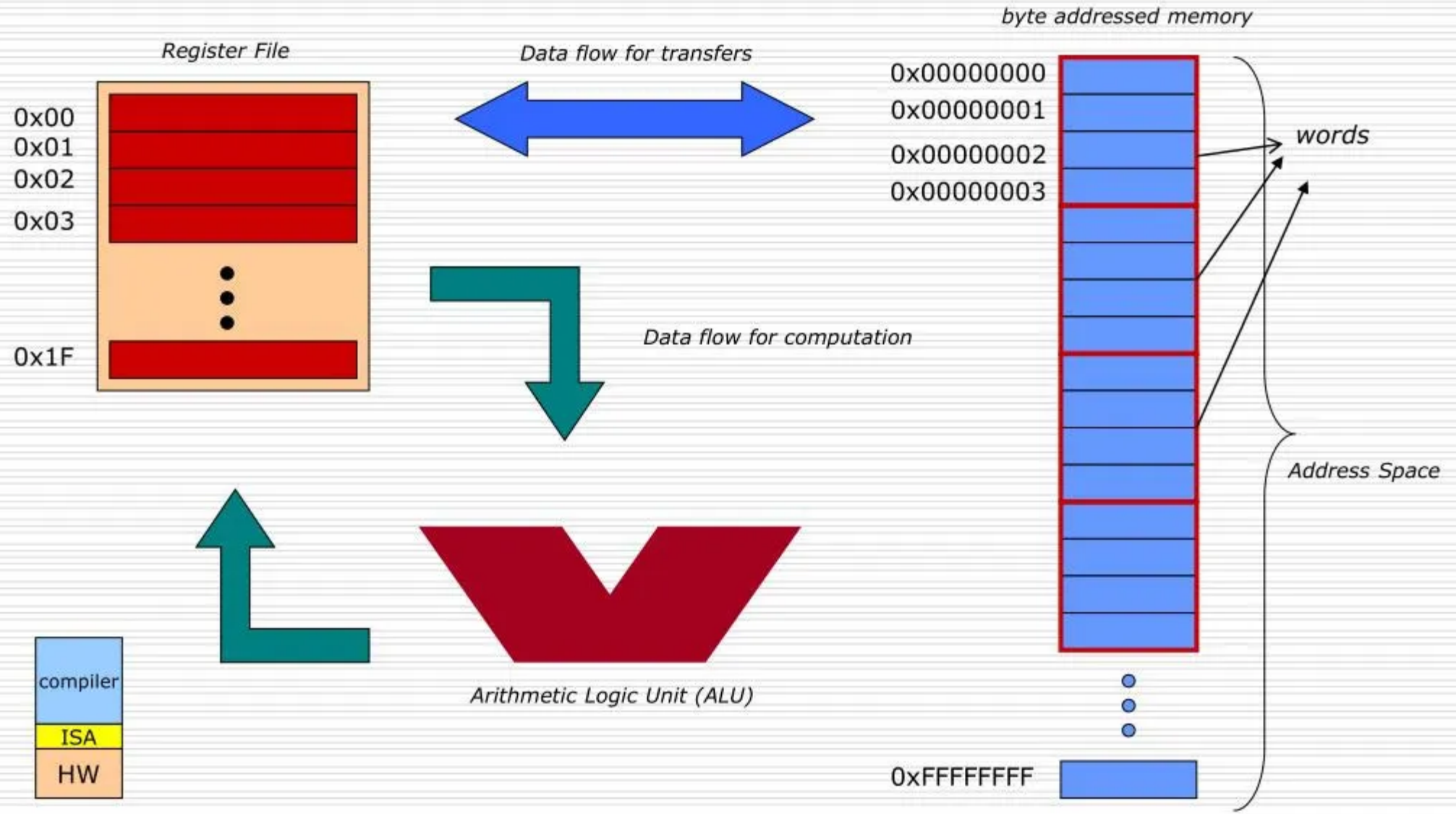
Q2. Execute code 4-1.v and 4-2.v

Explain the results differences.

RISC CPU Features

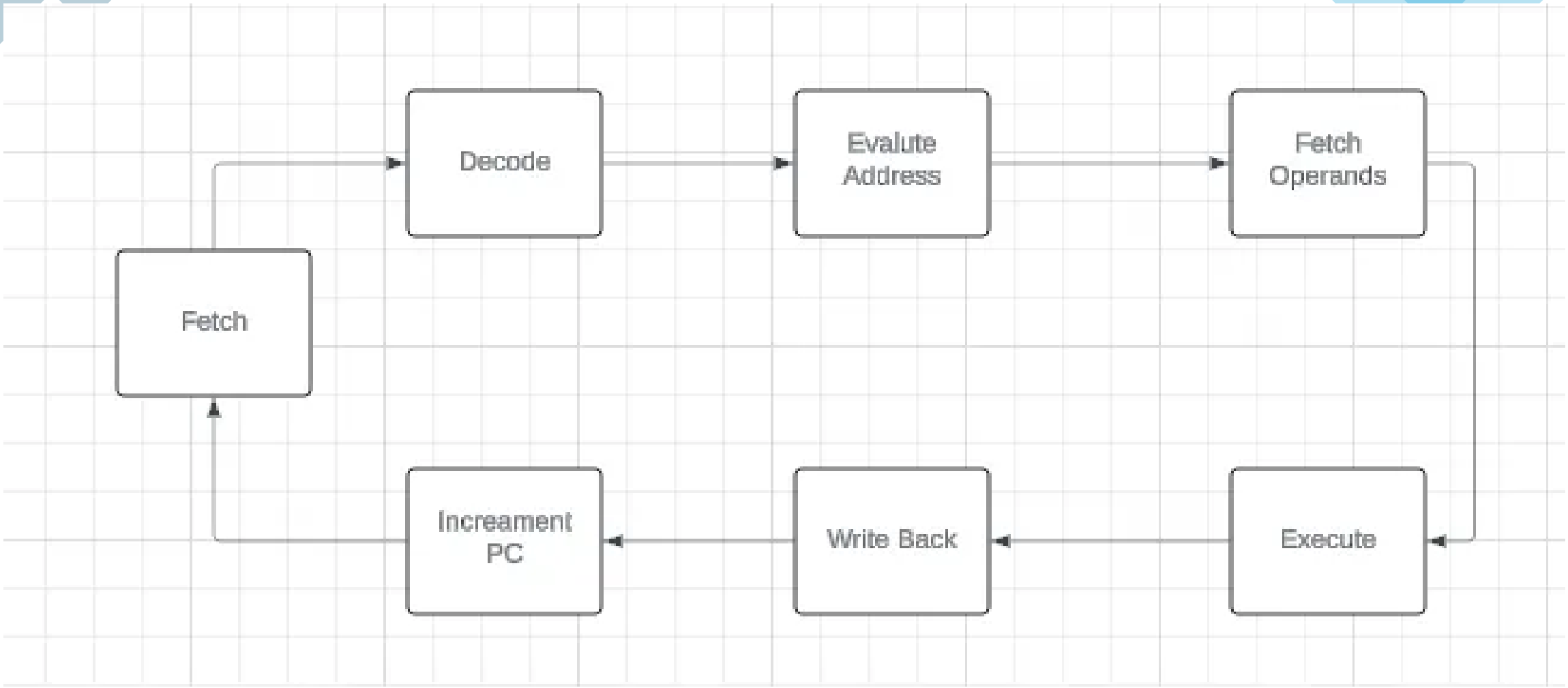
- RISC architectures focus on a smaller set of instructions, with each instruction performing a simple operation. RISC systems emphasize speed by executing instructions in a single clock cycle.
- Simplified instructions lead to faster execution. RISC processors are more energy-efficient.
- Requires more instructions per program, which may increase memory usage.
- It uses more registers and have complexity in compiler
- Examples are RISC-V, ARM...

Instruction Set Architecture (ISA)

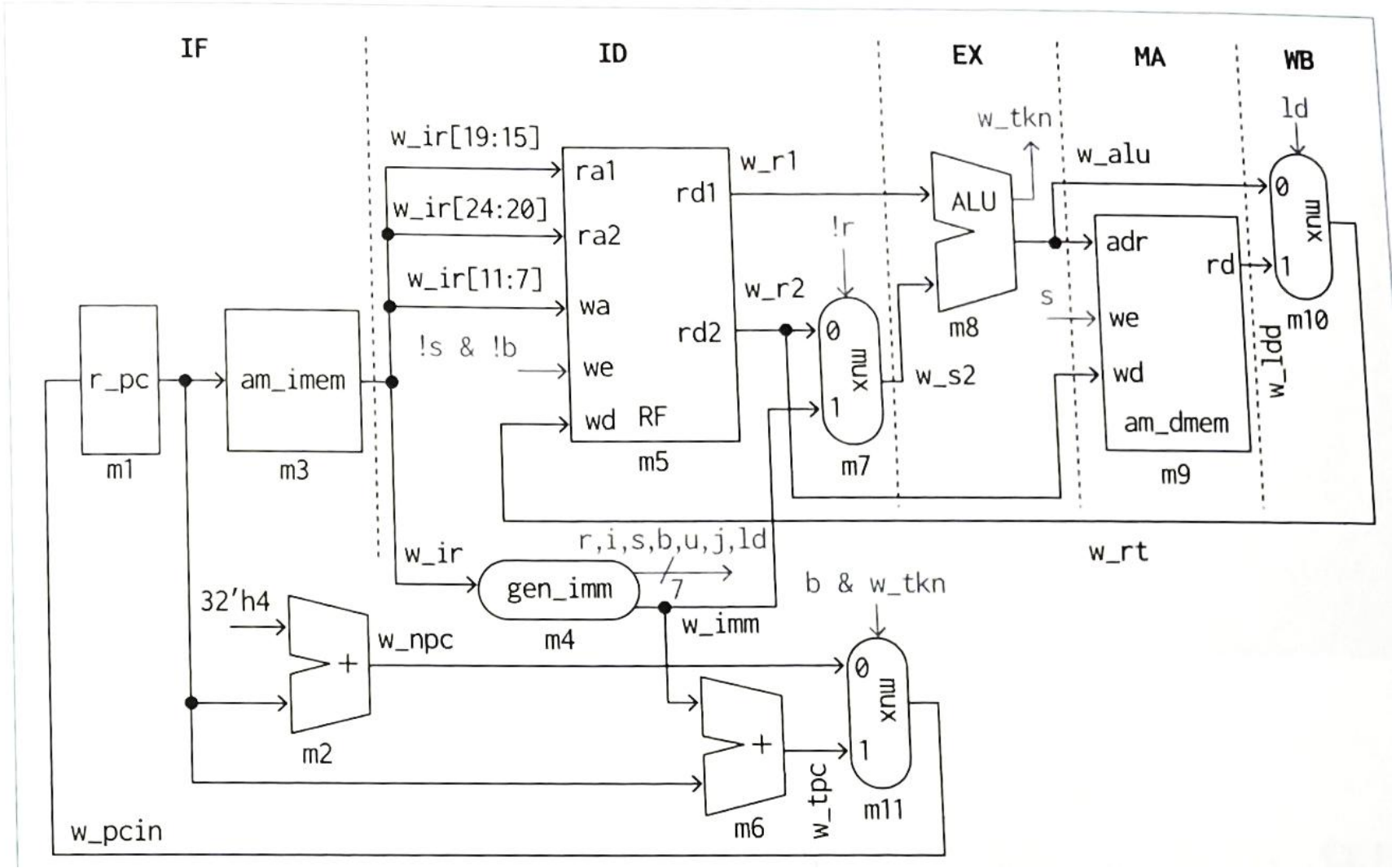


https://medium.com/@ranaumarnadeem/the-building-blocks-of-cpus-understanding-instruction-set-architecture-1f850e1dcb6a

Instruction Cycle

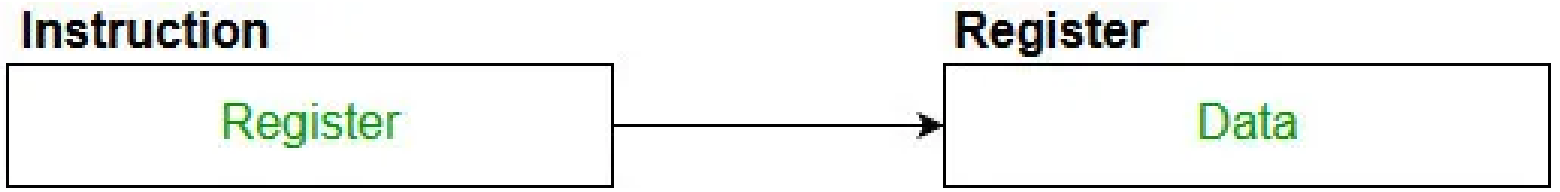


RISC-V Architecture we design

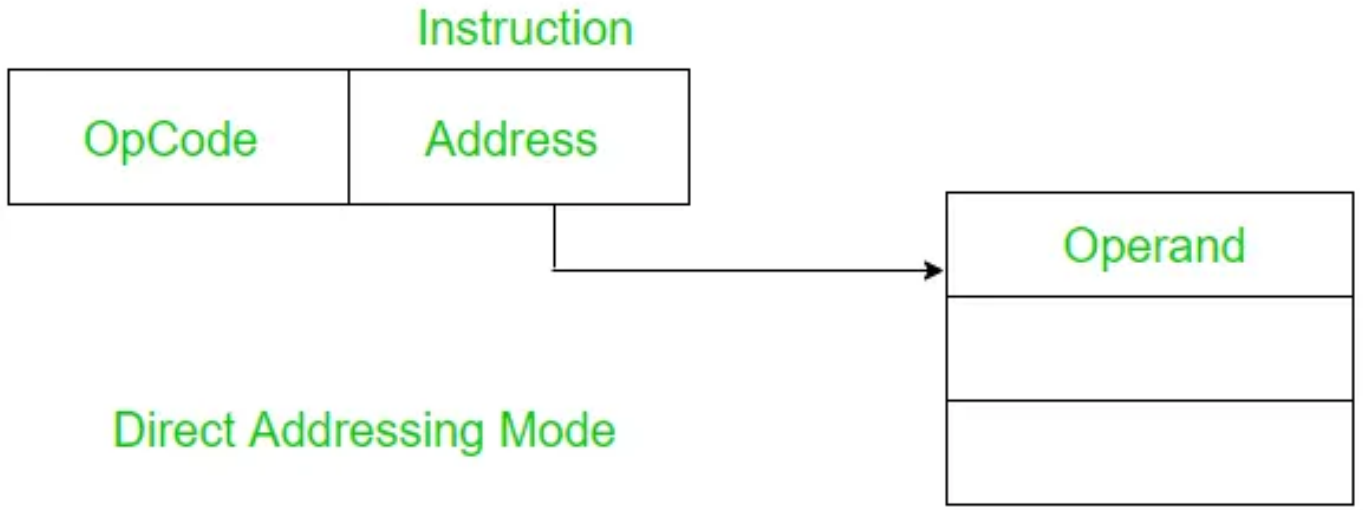


Addressing Modes

Immediate Addressing Mode



Register Addressing

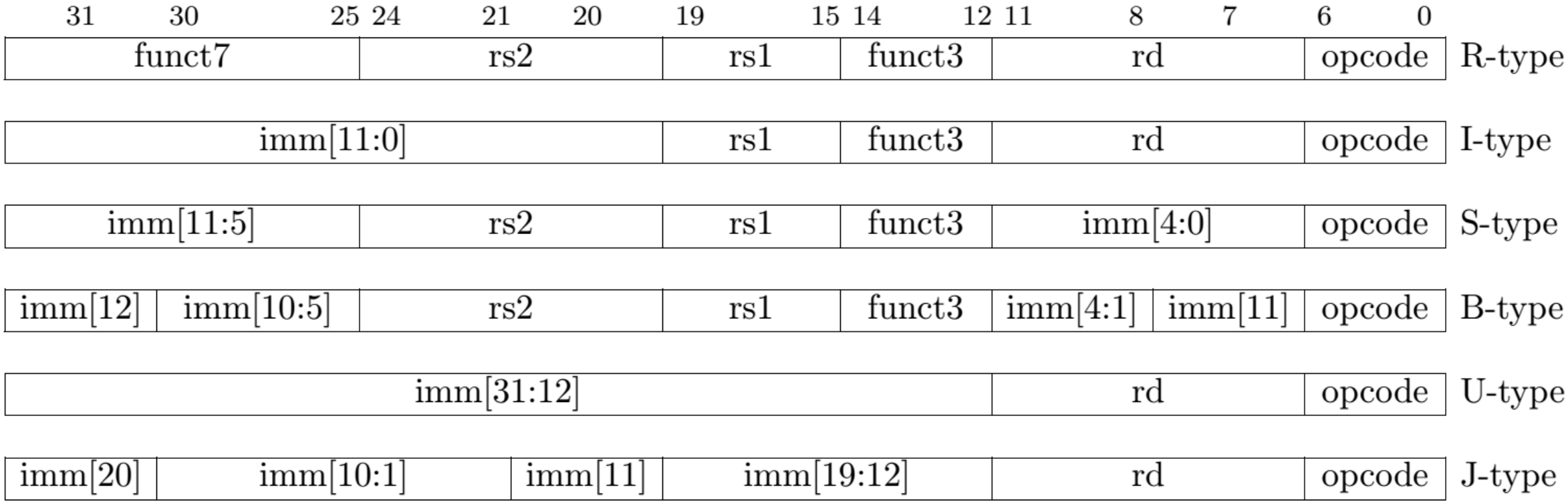


Direct Addressing Mode

Agenda

1-3	What is the RISC CPU?
4-1～4-8	Instruction types
5-1～5-2	Single Cycle Processor Components
5-3	Adder only
5-4	Register file
5-5～5-6	Immediate value, load, store, branch instructions

Type	Description	Main Usage
R	Register	Arithmetic/Logic (Register-Register)
I	Immediate	Load, Arithmetic (Register-Immediate)
S	Store	Store to Memory
B	Branch	Conditional Jumps
U	Upper	Large Immediate
J	Jump	Unconditional Jumps


31 30 25 24 21 20 19 15 14 12 11 8 7 6 0
funct7 rs2 rs1 funct3 rd opcode R-type
imm[11:0] rs1 funct3 rd opcode I-type
imm[11:5] rs2 rs1 funct3 imm[4:0] opcode S-type
imm[12] imm[10:5] rs2 rs1 funct3 imm[4:1] imm[11] opcode B-type
imm[31:12] rd opcode U-type
imm[20] imm[10:1] imm[11] imm[19:12] rd opcode J-type

Quiz 4-3

1. **m_get_type** of Code 4-3 is to get instruction type. Complete ***part. See slide 22 U-type.
2. Then execute module **m_top** of Code 4-4 code and show the result.
3. **m_get_imm** of Code 4-5 is to get immediate value. Complete *** part.

4-4 R type for arithmetic/logical/shift instruction

- Example of RISC-V assembly code for f = (a+b) –c

```
add x5, x2, x3     # x5 = a + b
sub x1, x5, x4     # f = x5 - c
```

R-Type Instruction

31 25	24 20	19 15	14 12	11 7	6 0	
funct7	rs2	rs1	funct3	rd	opcode	R-type
0000000	rs2	rs1	000	rd	0110011	R-type **add**
0100000	rs2	rs1	000	rd	0110011	R-type **sub**
0000000	rs2	rs1	001	rd	0110011	R-type **sll**
0000000	rs2	rs1	010	rd	0110011	R-type **slt**
0000000	rs2	rs1	011	rd	0110011	R-type **sltu**
0000000	rs2	rs1	100	rd	0110011	R-type **xor**
0000000	rs2	rs1	101	rd	0110011	R-type **srl**
0100000	rs2	rs1	101	rd	0110011	R-type **sra**
0000000	rs2	rs1	110	rd	0110011	R-type **or**
0000000	rs2	rs1	111	rd	0110011	R-type **and**

Quiz 4-4 R-type

- Show R-type instruction bit series in terms of the following assembly code ;

  add x1, x2, x3

Answer:

```
32'b0000000_00011_00010_000_00001_01100_11
```

4-5 I-type instruction

- Example of RISC-V assembly code for f = (a+b) +9

```
add  x5, x2, x3     # x5 = a + b
addi x1, x5, 9      # f = x5 + 9
```

4-5 I-type instruction

31 25	24 20	19 15	14 12	11 7	6 0	
imm[11:0]		rs1	funct3	rd	opcode	I-type
imm[11:0]		rs1	000	rd	0010011	I-type **addi**
imm[11:0]		rs1	010	rd	0010011	I-type **slti**
imm[11:0]		rs1	011	rd	0010011	I-type **sltiu**
imm[11:0]		rs1	100	rd	0010011	I-type **xori**
imm[11:0]		rs1	110	rd	0010011	I-type **ori**
imm[11:0]		rs1	111	rd	0010011	I-type **andi**
0000000	shamt	rs1	001	rd	0010011	I-type **slli**
0000000	shamt	rs1	101	rd	0010011	I-type **srli**
0100000	shamt	rs1	101	rd	0010011	I-type **srai**

Quiz 4-5 I-type

- Show I-type instruction bit series in terms of the following assembly code ;

  addi x3, x4, 7

Answer: 32'b000000000111_00100_000_00011_01100_11

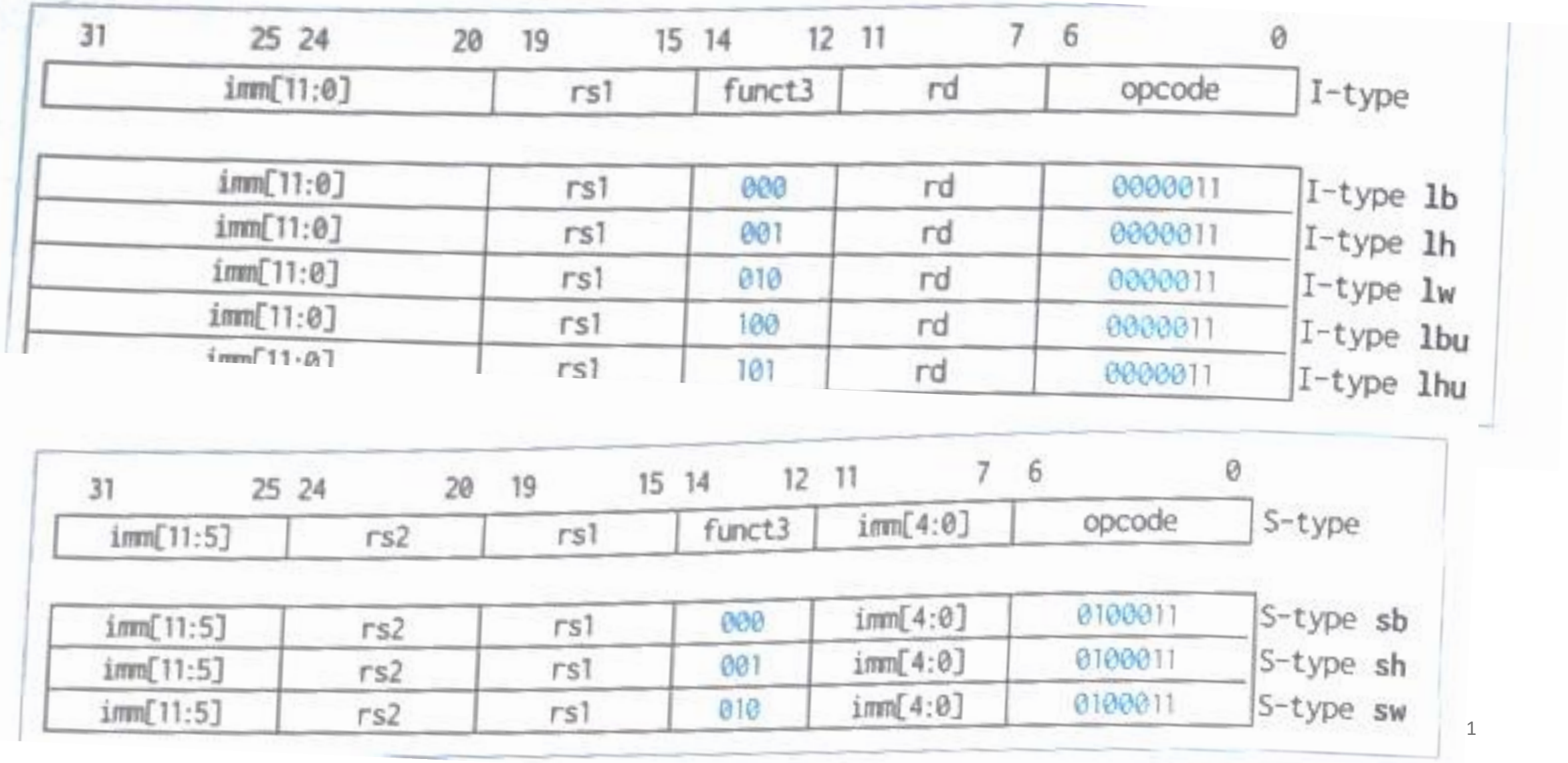
4-6 Load, Store instruction

- Show an RISC-V assembly language program that loads 32-bit data stored at main memory addresses 512 and 516 into x1 and x2 respectively.

  Then stores the result of their addition in x3. Use x0, which holds the value 0, for address calculations.

```
lw x1, 512(x0)
lw x2, 516(x0)
add x3, x1, x2
```

4-6 Load, Store instruction



31 25	24 20	19 15	14 12	11 7	6 0	
imm[11:0]		rs1	funct3	rd	opcode	I-type
imm[11:0]		rs1	000	rd	0000011	I-type lb
imm[11:0]		rs1	001	rd	0000011	I-type lh
imm[11:0]		rs1	010	rd	0000011	I-type lw
imm[11:0]		rs1	100	rd	0000011	I-type lbu
imm[11:0]		rs1	101	rd	0000011	I-type lhu

31 25	24 20	19 15	14 12	11 7	6 0	
imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode	S-type
imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	S-type sb
imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	S-type sh
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	S-type sw

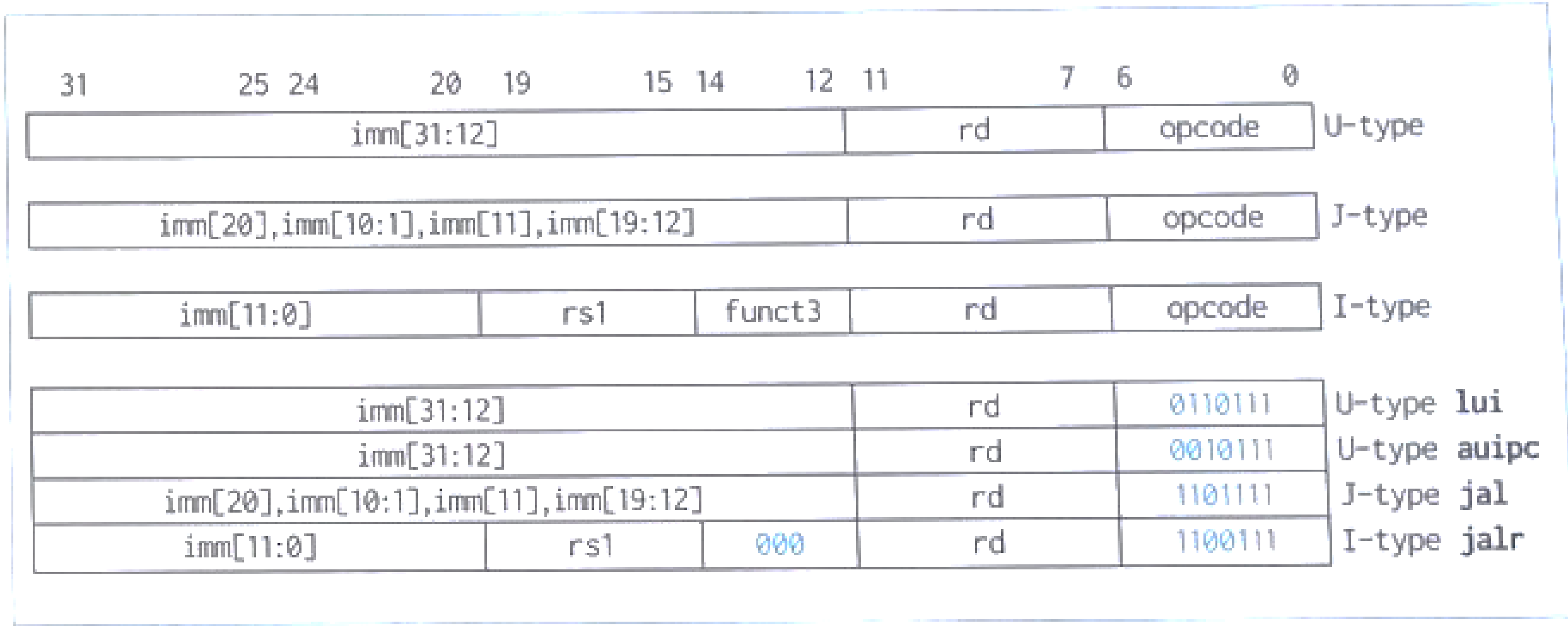
4-7 Branch instruction and Program Counter

```
32'h0   addi x5, x0, 1     # x5 = 1
32'h4   addi x6, x0, 2     # x6 = 2
32'h8   add  x7, x5, x6    # x7 = x5 + x6
```

31 25	24 20	19 15	14 12	11 7	6 0	
imm[12],imm[10:5]	rs2	rs1	funct3	imm[4:1],imm[11]	opcode	B-type
imm[12],imm[10:5]	rs2	rs1	000	imm[4:1],imm[11]	1100011	B-type **beq**
imm[12],imm[10:5]	rs2	rs1	001	imm[4:1],imm[11]	1100011	B-type **bne**
imm[12],imm[10:5]	rs2	rs1	100	imm[4:1],imm[11]	1100011	B-type **blt**
imm[12],imm[10:5]	rs2	rs1	101	imm[4:1],imm[11]	1100011	B-type **bge**
imm[12],imm[10:5]	rs2	rs1	110	imm[4:1],imm[11]	1100011	B-type **bltu**
imm[12],imm[10:5]	rs2	rs1	111	imm[4:1],imm[11]	1100011	B-type **bgeu**

imm[12:2]	imm[12:1]	imm[12:1]	{imm[12],imm[10:5],imm[4:1],imm[11]}
-10	-20	12'b111111101100	12'b1111110_11001
-9	-18	12'b111111101110	12'b1111110_11101
-8	-16	12'b111111110000	12'b1111111_00001
-7	-14	12'b111111110010	12'b1111111_00101
-6	-12	12'b111111110100	12'b1111111_01001
-5	-10	12'b111111110110	12'b1111111_01101
-4	-8	12'b111111111000	12'b1111111_10001
-3	-6	12'b111111111010	12'b1111111_10101
-2	-4	12'b111111111100	12'b1111111_11001
-1	-2	12'b111111111110	12'b1111111_11101
0	0	12'b000000000000	12'b0000000_00000
1	2	12'b000000000010	12'b0000000_00100
2	4	12'b000000000100	12'b0000000_01000
3	6	12'b000000000110	12'b0000000_01100
4	8	12'b000000001000	12'b0000000_10000

4-8 lui, auipc, jal, jalr and other instructions



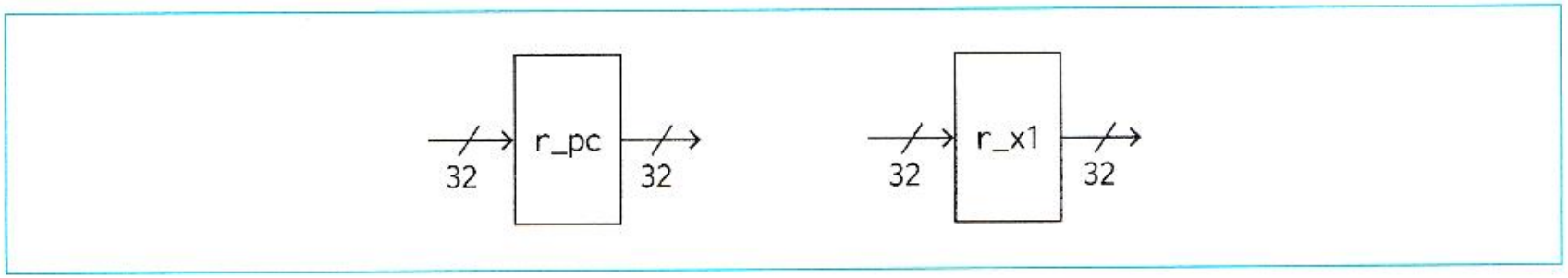
Quiz 4-7　x10?

```
32'h0            addi x10, x0, 0
32'h4            addi x3, x0, 0
32'h8            addi x1, x0, 11
32'hc    label:  add  x10, x10 , x3
32'h10           addi x3, x3,1
32'h14           bne  x3, x1, label
```

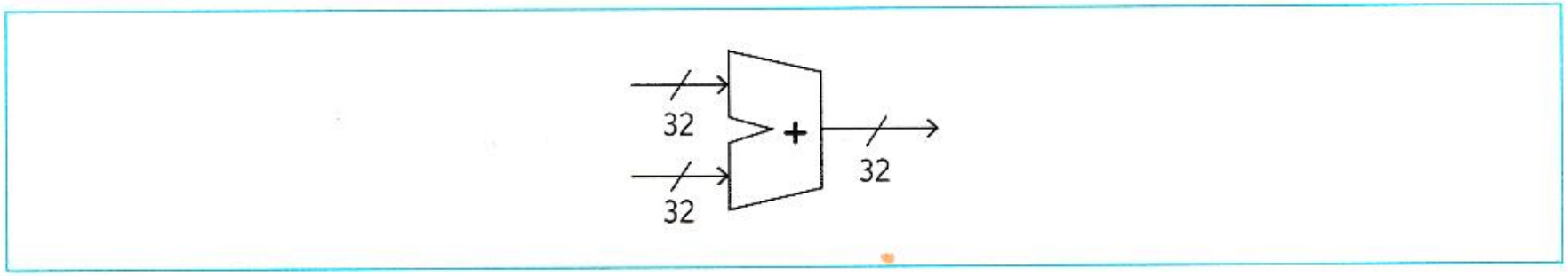
Agenda

1-3	What is the RISC CPU?
4-1～4-8	Instruction types
5-1～5-2	Single Cycle Processor Components
5-3	Adder only
5-4	Register file
5-5～5-6	Immediate value, load, store, branch instructions

Program counter: Register / Adder

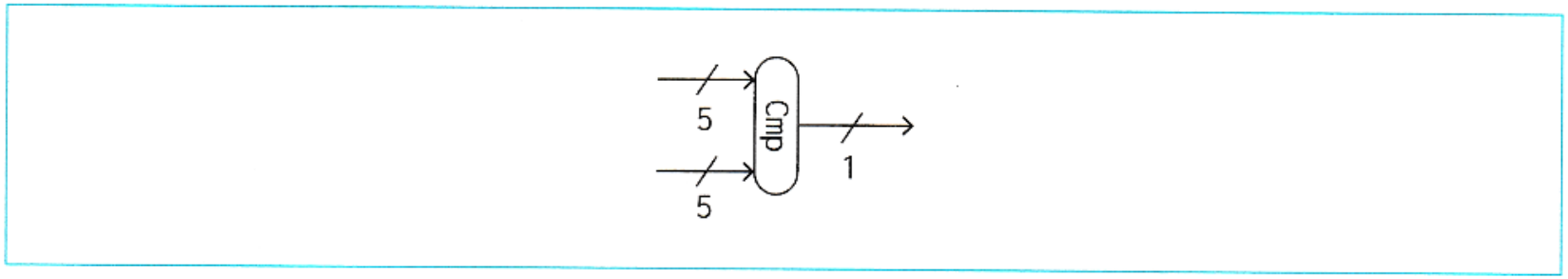


Register

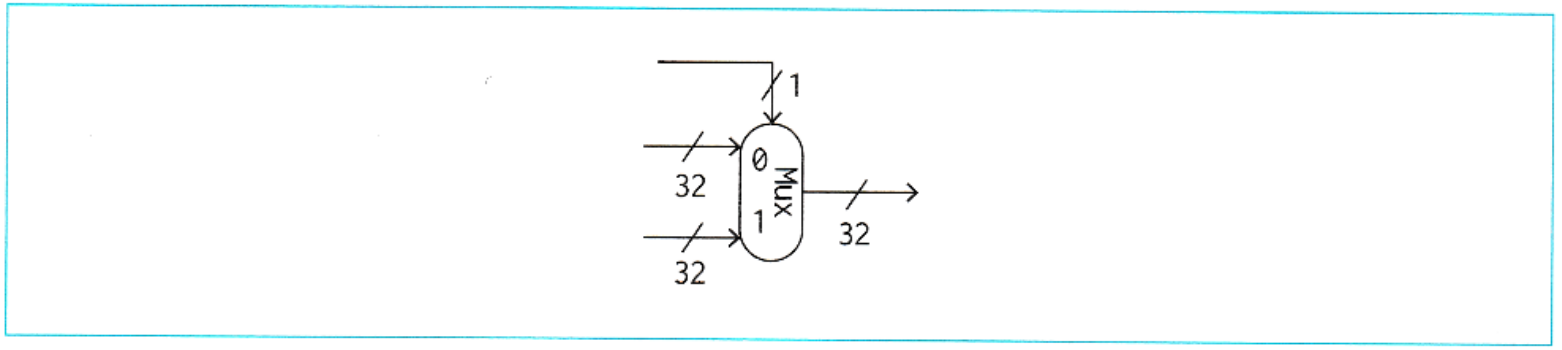


Adder: Code 5-1

Comparator / Multiplexer

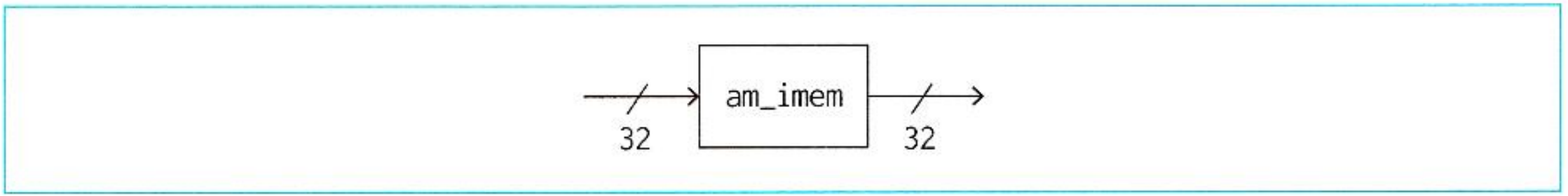


Comparator: Code 5-2



Multiplexer: Code 5-3 Quiz

Memory



Asynchronous memory: Code 5-4

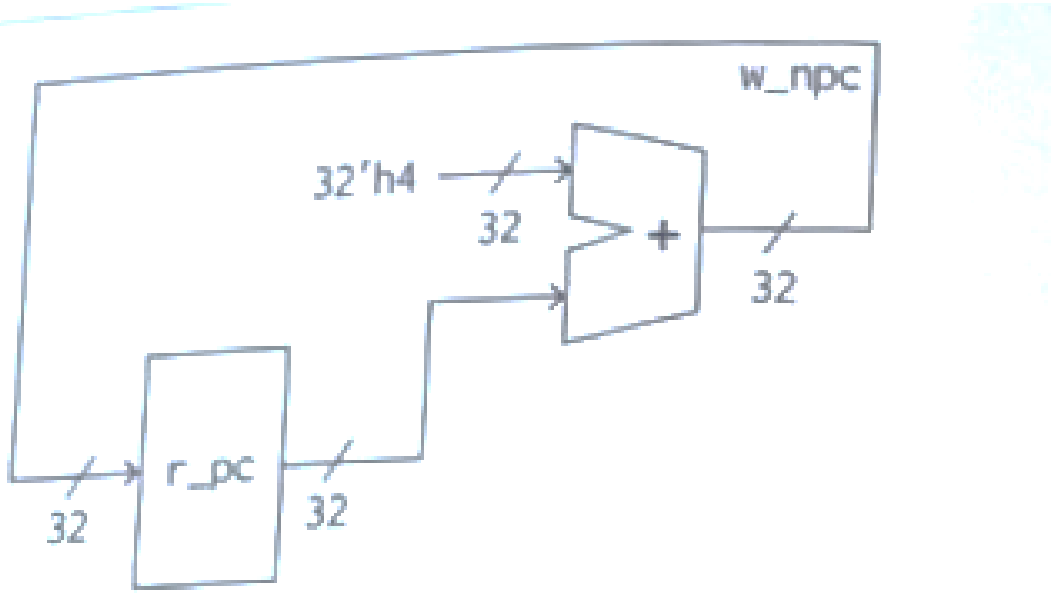
```
32'h0      add x1, x0, x1   # x1 = 0 + 3
32'h4      add x1, x1, x0   # x1 = 3 + 0
32'h8      add x1, x1, x1   # x1 = 3 + 3
```

Agenda

1-3	What is the RISC CPU?
4-1～4-8	Instruction types
5-1～5-2	Single Cycle Processor Components
5-3	Adder only architecture
5-4	Register file
5-5～5-6	Immediate value, load, store, branch instructions

Adder + PC

m_circuit1
Code 5-5

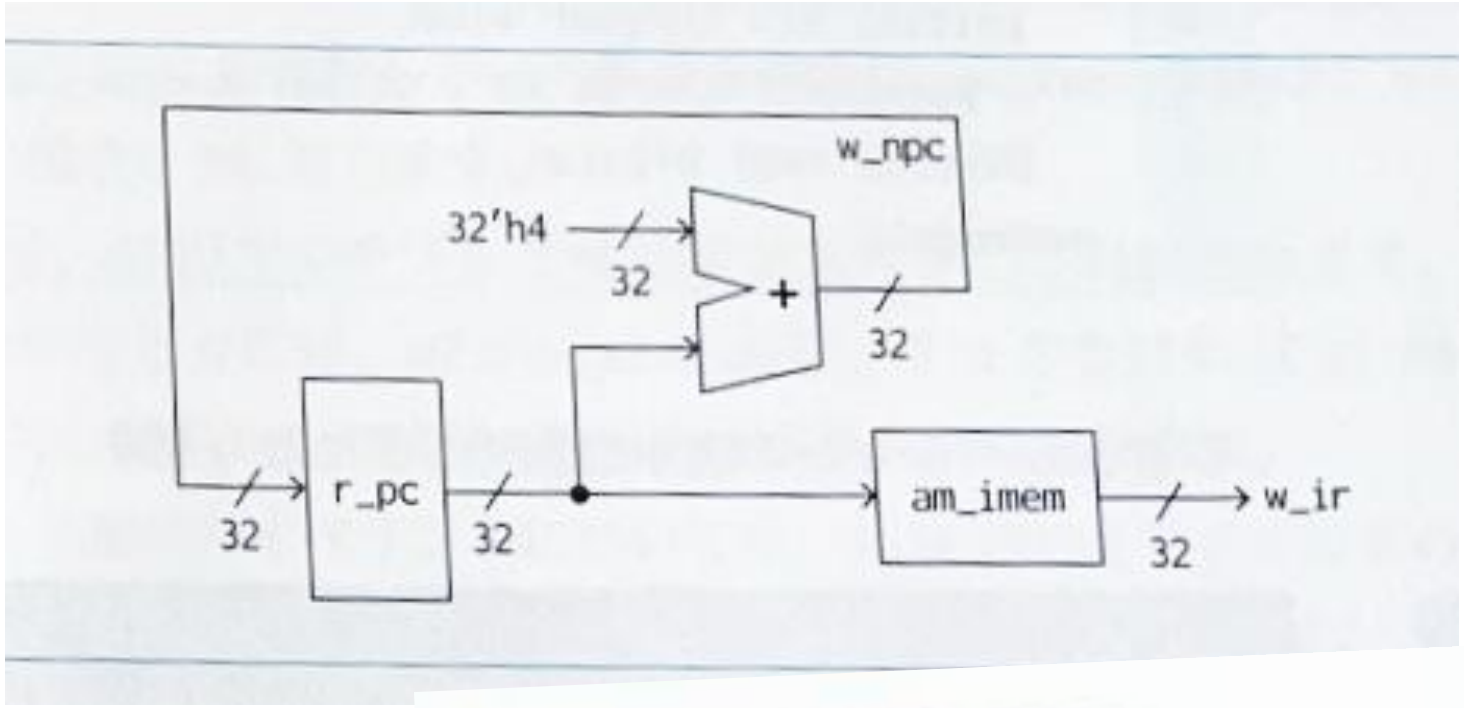



```
199 00000000
299 00000004
399 00000008
499 0000000c
```

Code 5-6 result; Quiz

+ ROM memory (am_imem)

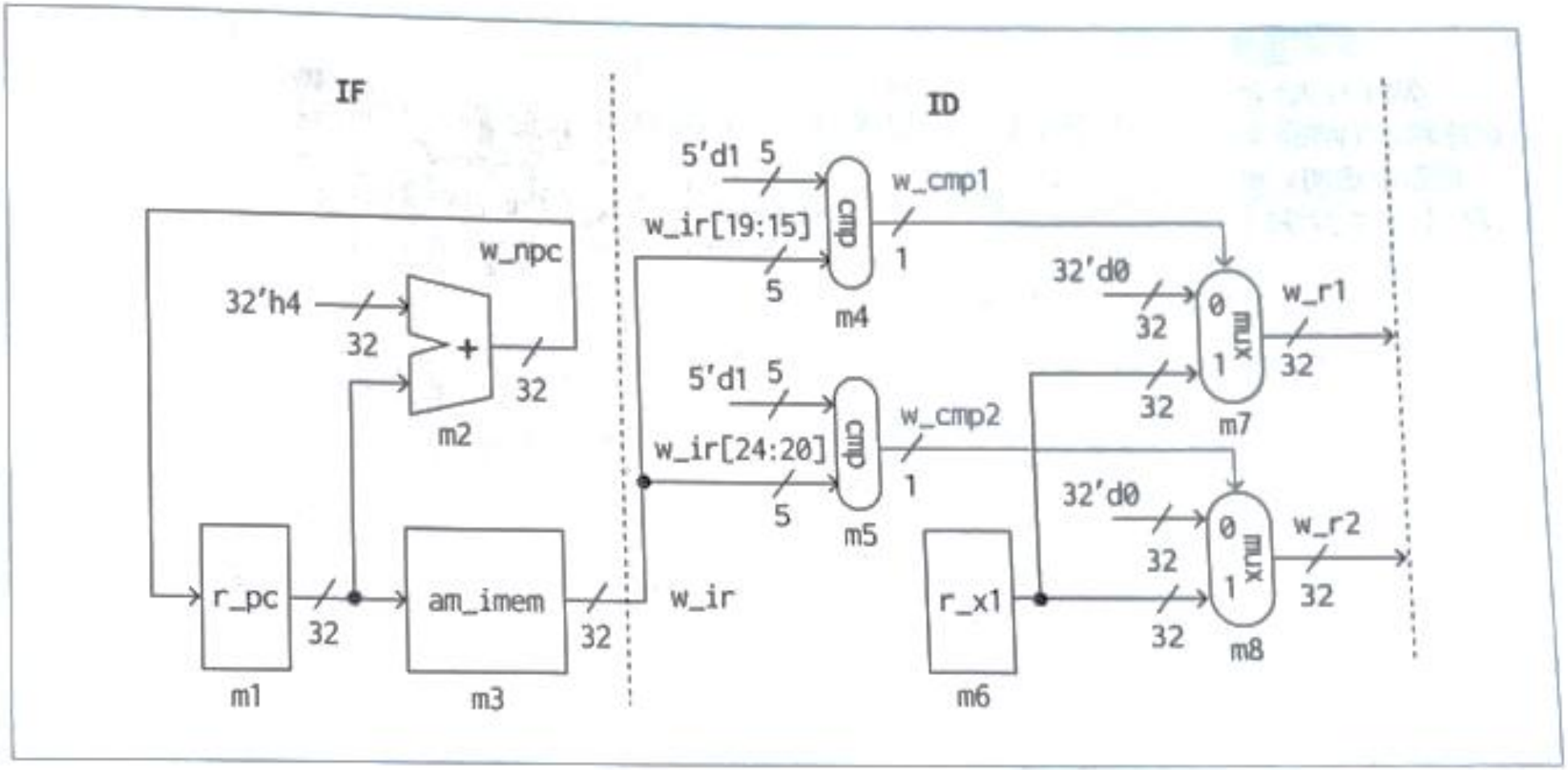
m_circuit2
Code 5-7



Code 5-8 result; Quiz

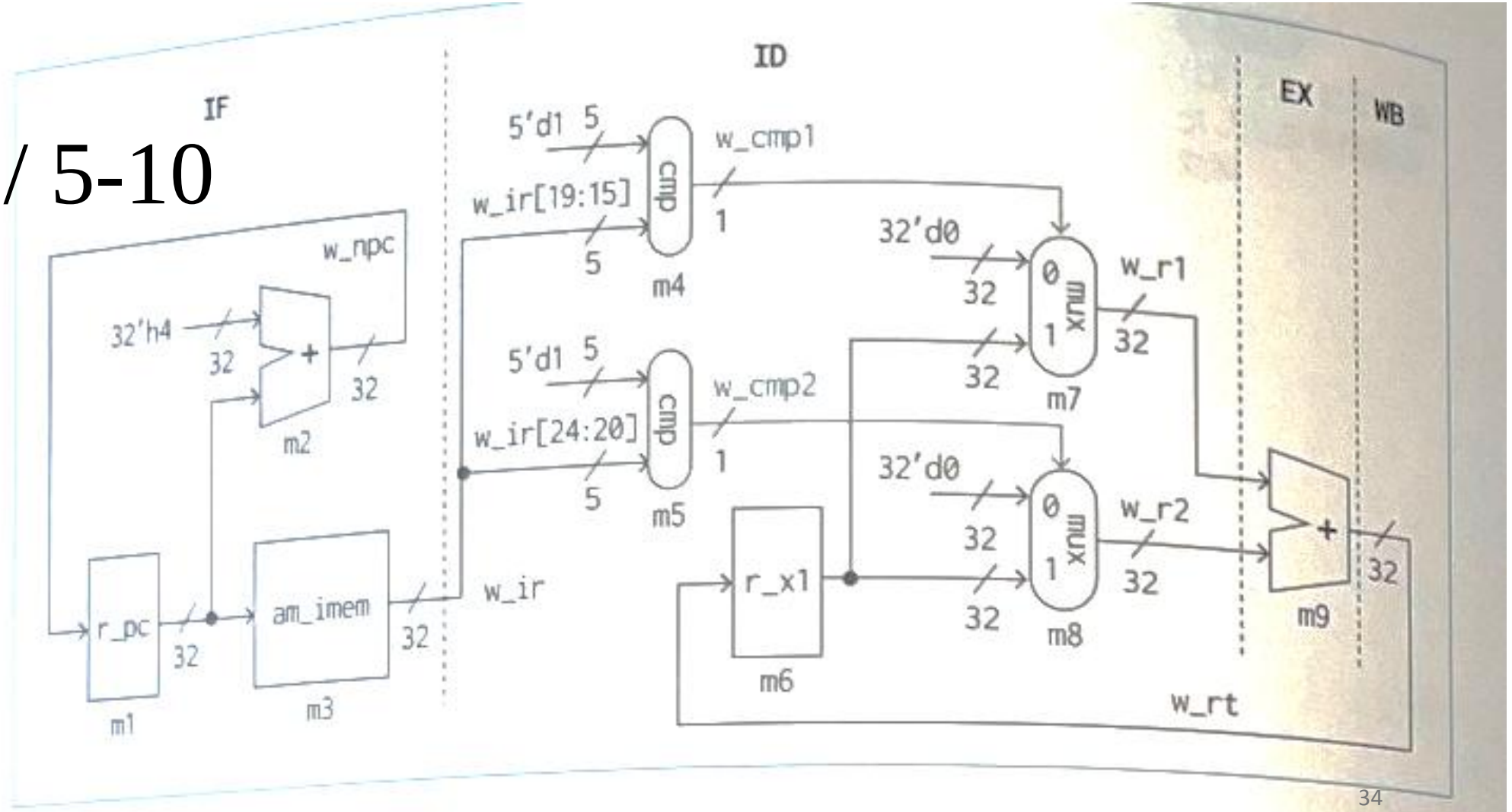
```
199 00000000 001000b3
299 00000004 000080b3
399 00000008 001080b3
```

+ Instruction Decoder (ID)



+ EX and WB

m_proc1
Code 5-9/ 5-10

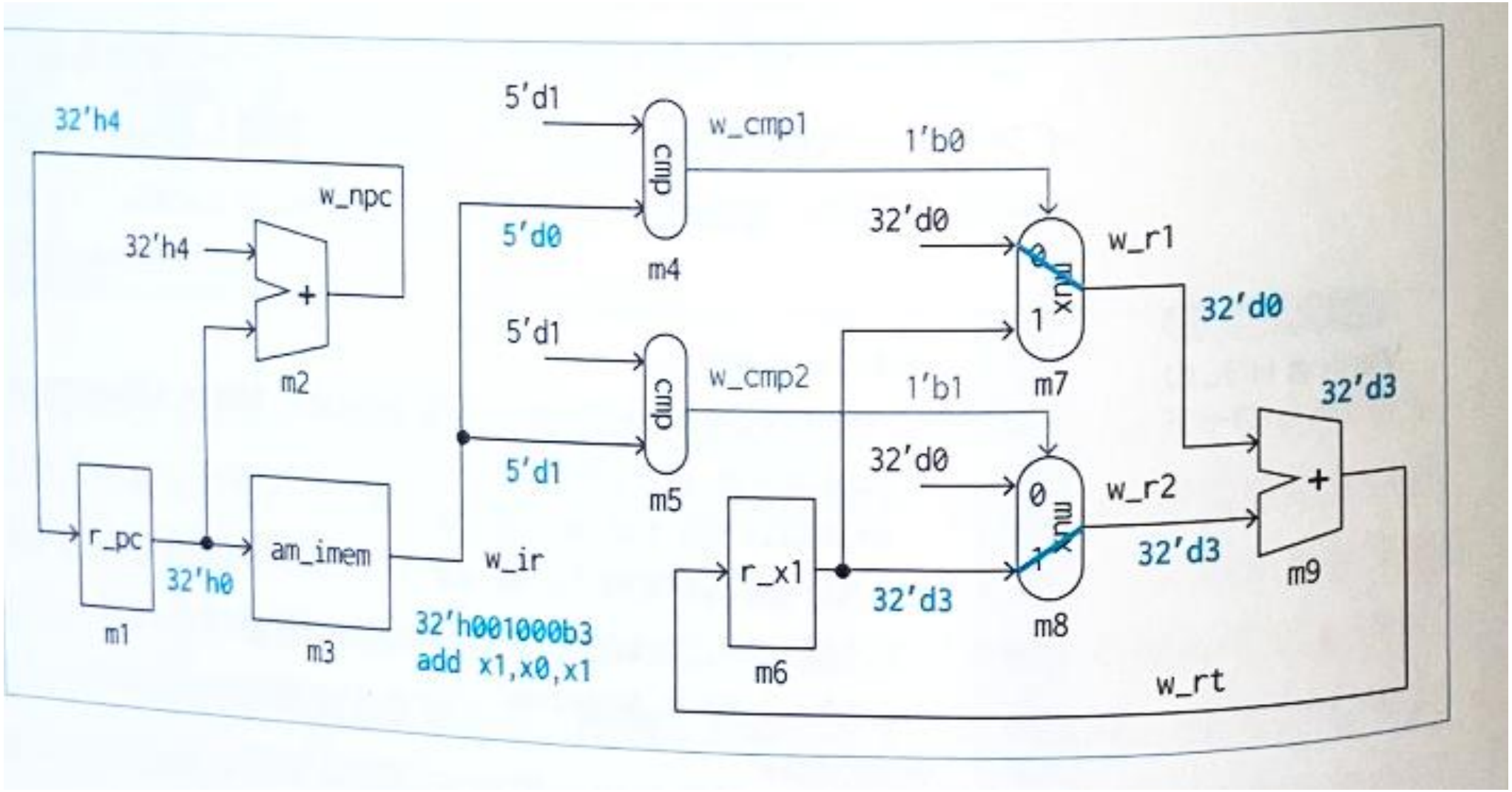


Code 5-11 result; Quiz

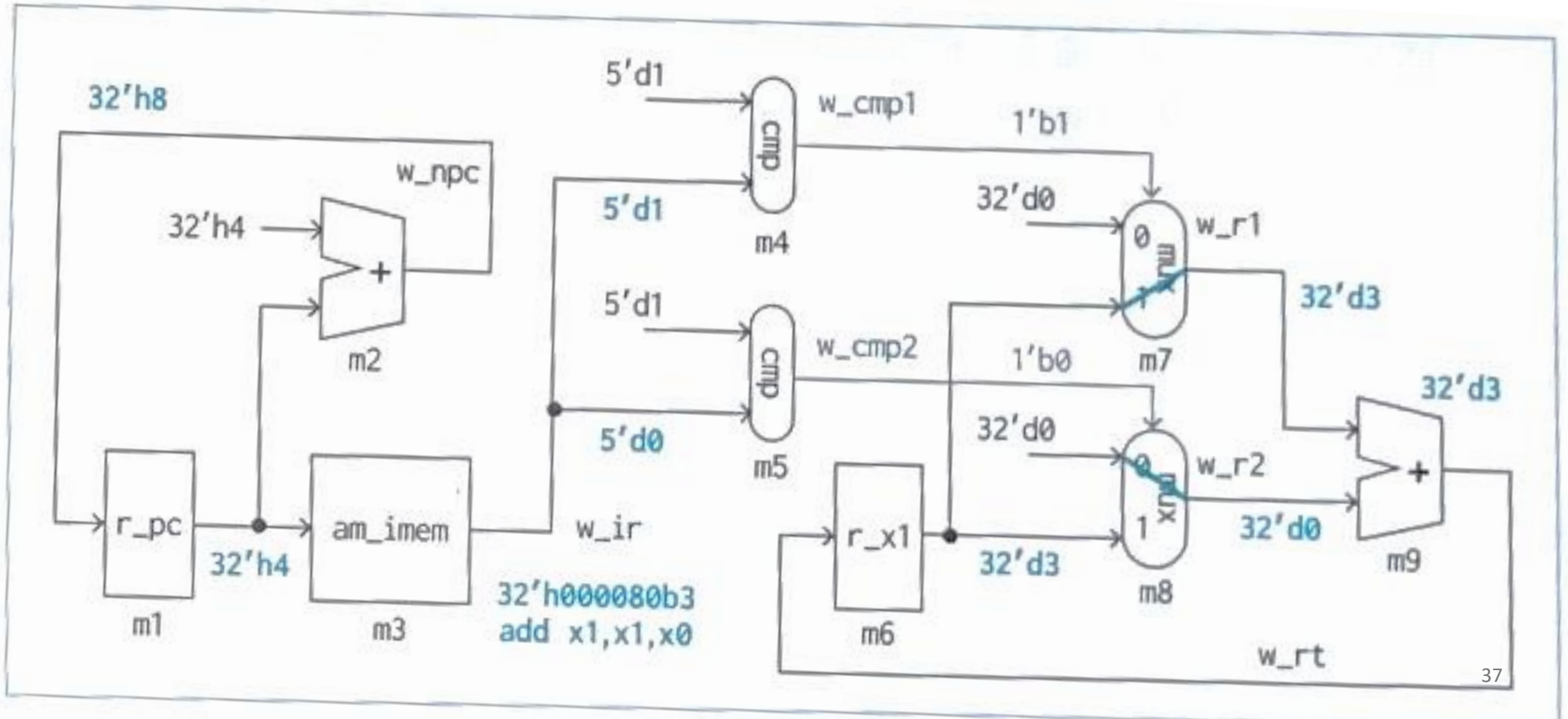
time	w_r1	w_r2	w_rt
199	0	3	3
299	3	0	3
399	3	3	6

```
32'h0      add x1, x0, x1   # x1 = 0 + 3
32'h4      add x1, x1, x0   # x1 = 3 + 0
32'h8      add x1, x1, x1   # x1 = 3 + 3
```

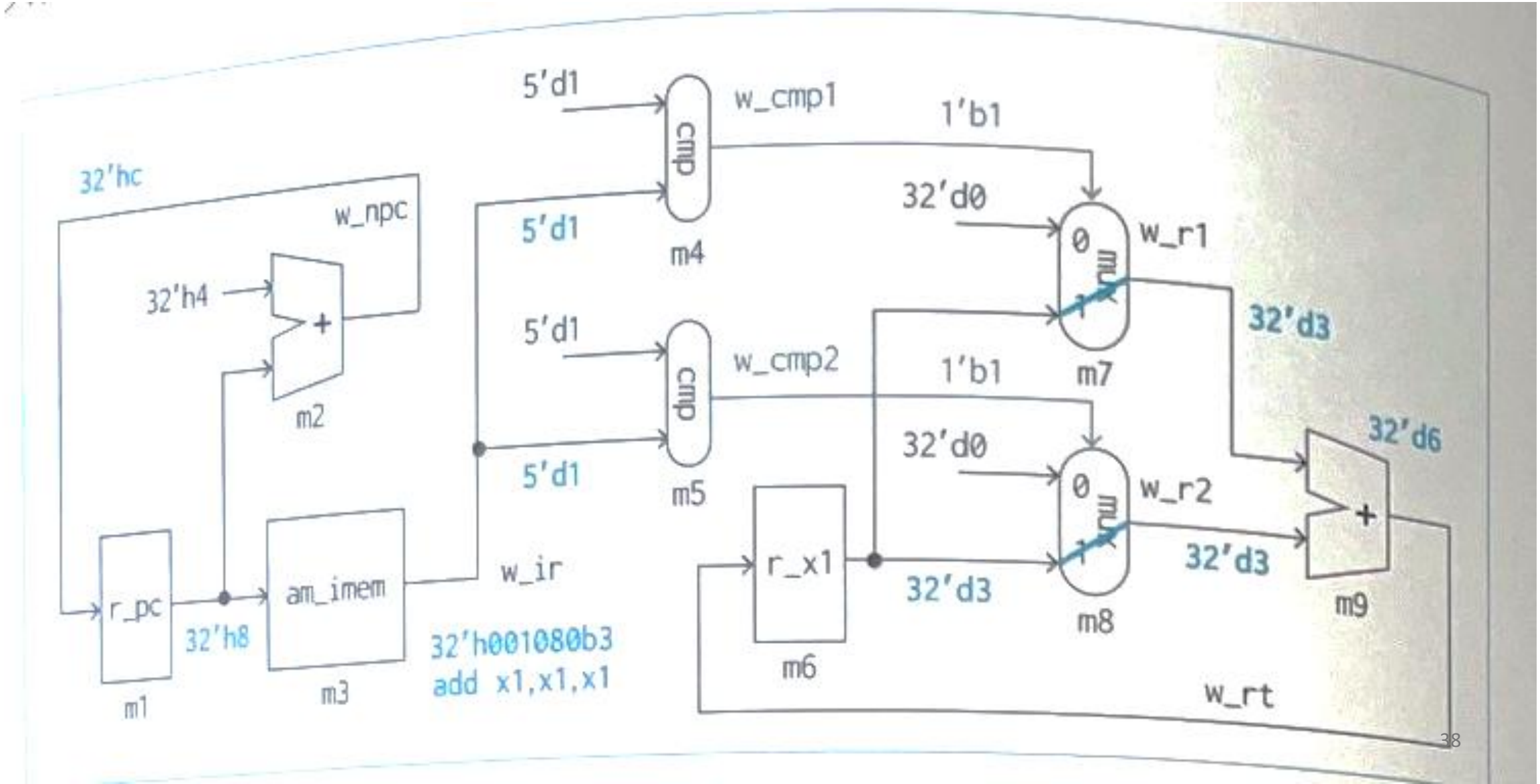
Cycle Clock 1 : CC1



CC2


32'h8
w_npc
32'h4
m2
r_pc
32'h4
m1
am_imem
m3
w_ir
32'h000080b3
add x1,x1,x0
5'd1
cmp
w_cmp1
m4
5'd1
5'd1
cmp
w_cmp2
m5
5'd0
1'b1
32'd0
0
mux
1
w_r1
m7
32'd3
1'b0
32'd0
0
mux
1
w_r2
m8
32'd3
r_x1
m6
32'd3
32'd3
+
m9
32'd0
w_rt

CC3

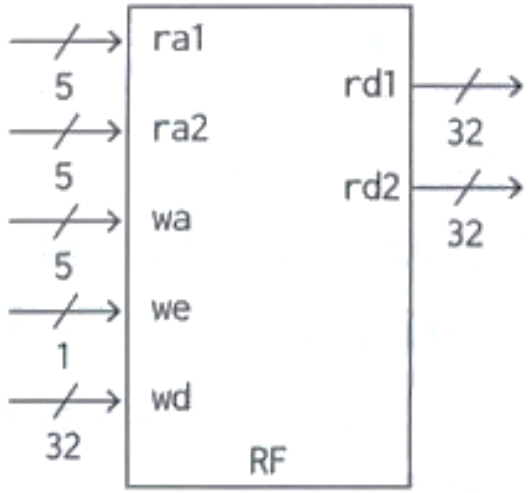

5'd1
w_cmp1
1'b1
cmp
32'hc
w_npc
32'd0
5'd1
m4
0
m
1
w_r1
32'h4
+
32'd3
5'd1
w_cmp2
1'b1
m7
m2
cmp
32'd6
32'd0
0
m
1
w_r2
5'd1
m5
+
r_x1
32'd3
32'd3
m9
m8
r_pc
am_imem
w_ir
32'h8
32'h001080b3
add x1,x1,x1
m6
m1
m3
w_rt

Agenda

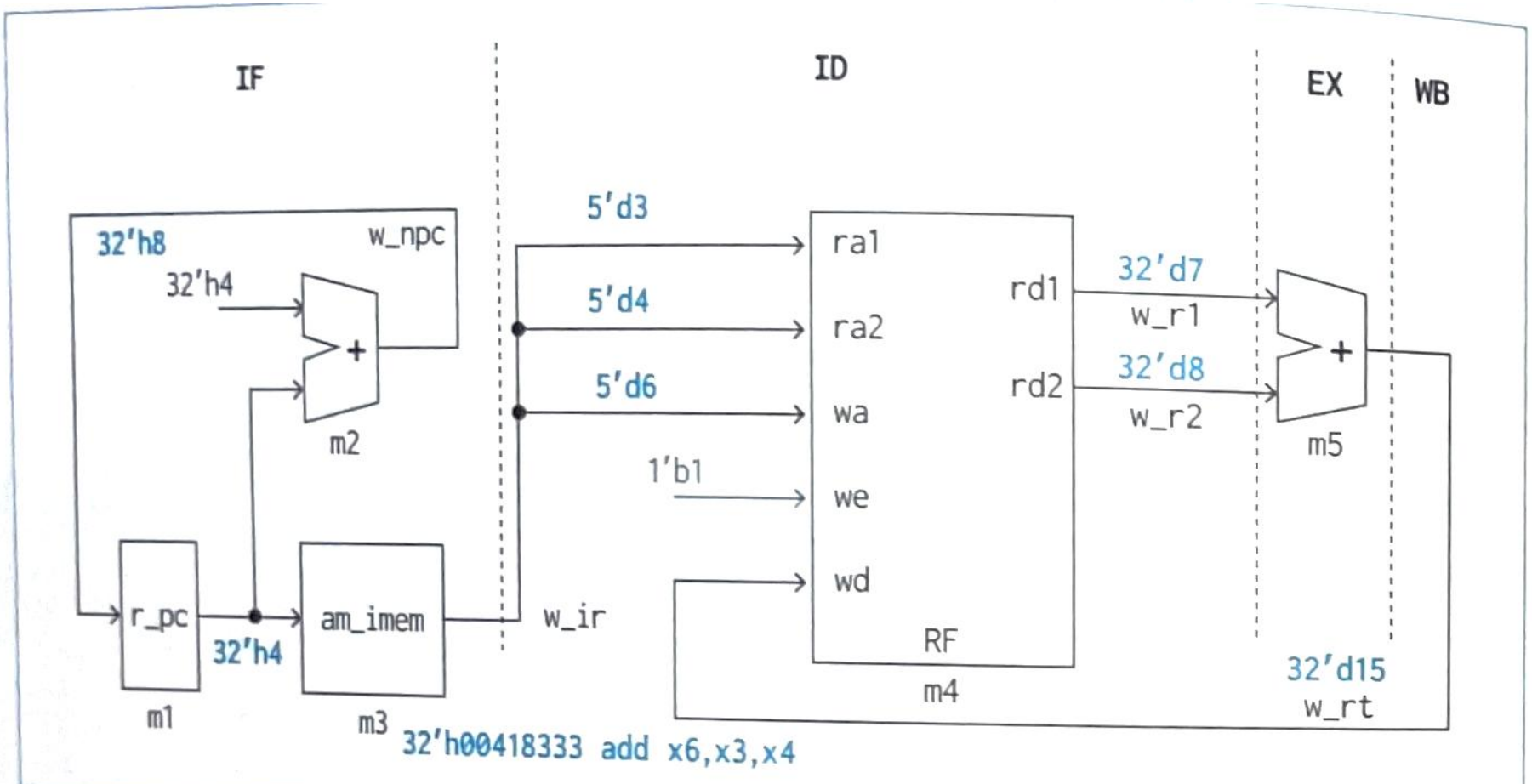
1-3	What is the RISC CPU?
4-1～4-8	Instruction types
5-1～5-2	Single Cycle Processor Components
5-3	Adder
5-4	Register file
5-5～5-6	Immediate value, load, store, branch instructions

+ Register file: m_RF

Code 5-12



m_proc2
Code 5-13



Code 5-14, 5-15 result; Quiz

time	w_r1	w_r2	w_rt
199	5	6	11
299	7	8	15
399	11	15	26

```
add x5,x1,x2
add x6,x3,x4
add x7,x5,x6
```

Agenda

1-3	What is the RISC CPU?
4-1～4-8	Instruction types
5-1～5-2	Single Cycle Processor Components
5-3	Adder
5-4	Register file
5-5～5-6	Immediate value, load, store, branch instructions